PyTorch

```
# 1. Tensor Datatypes
# float32, float64, int64, bool
a = torch.tensor([1, 2, 3], dtype=torch.float32)

# 2. Check dtype:
print(a.dtype)

# 3. Creating Tensor
a = torch.empty(2,3)
a = torch.zeros(2,3)
a = torch.ones(2,3)
a = torch.rand(2,3)
a = torch.empty(2,3)
    .normal_(mean=0, std=1)
a = torch.empty(2,3)
        .uniform_(0,1)
a = torch.diag(torch.ones(3))

# 4. Using Tensor
a = torch.tensor([[1,2], [3,4]])
a = torch.arange(0,10,2)
a = torch.linspace(0,10,4)
a = torch.eye(5) # identity matrix of 5x5
a = torch.full((3,3), 5)

# 5. Like-functions
a = torch.rand(2,3)
torch.empty_like(a)
torch.zeros_like(a)
torch.ones_like(a)
torch.rand_like(a)

# 6. Clamp
a = torch.tensor([-3, 0, 5, 10])
torch.clamp(a, min=2, max=3)
[2.0, 2.3, 3.0, 3.0]

# 7. argmax & argmin
max -> gives VALUE
argmax -> gives POSITION (index)
scores = torch.tensor([0.1, 2.5, 0.3])
prediction = torch.argmax(scores)
torch.argmax(a, dim=0) # column-wise
torch.argmax(a, dim=1) # row-wise

Ex:
output.shape = [batch_size, num_classes]
output = torch.tensor([
  [0.1, 2.5, 0.3],
  [4.0, 1.2, 0.5]
])
pred = torch.argmax(output, dim=1)
print(pred) # tensor([1,0])
Meaning: sample 1 → class 1
         sample 2 → class 0
```

```
# Important tensor properties
a.shape
a.size()
a.ndim
a.numel()

# clamp
a = torch.tensor([-3, 0, 5, 10])
b = torch.clamp(a, min=0, max=6) #[0, 0, 5, 6]
```

rand() vs randn()

```
torch.rand(3) # [0.12, 0.77, 0.43]
torch.randn(5) # Mean ≈ 0, stdDev ≈ 1
```

view() and reshape()

view(): Requires contiguous memory **(same memory)**

```
a = torch.arange(6).view(2,3) # [[0,1,2],[3,4,5]]
b = a.t() # transpose : [[0,3],[1,4], [2,5]]
print(b.is_contiguous()) # False
b.view(6) # RuntimeError: view size is not compatible
```

reshape() is smarter.

If tensor is contiguous: ➔ behaves like view()
If not contiguous: ➔ creates a **copy** automatically

```
a = torch.arange(12).reshape(3,4)
```

squeeze() vs unsqueeze()

unsqueeze() → add a dimension

```
a = torch.tensor([1,2,3])
print(a.shape) # torch.Size([3])

b = a.unsqueeze(0)
print(b.shape) # torch.Size([1,3])
print(b) # [[1,2,3]]
```

squeeze() → remove dimensions of size 1

```
c = b.squeeze()
print(c.shape) # [3]
```

Why needed?

Sometimes models output extra useless dimensions.
Example: image.shape = [3,224,224]
Model expects: [batch, channels, height, width]
So: image = image.unsqueeze(0)
Now: [1,3,224,224]

GPU Performance Comparison

```
import torch
import time

# Check if GPU is available
device = "cuda" if torch.cuda.is_available() else "cpu"

print("GPU Available:", torch.cuda.is_available())

# Matrix size
SIZE = 4000

# Create matrices for CPU
a_cpu = torch.rand(SIZE, SIZE)
b_cpu = torch.rand(SIZE, SIZE)

# ---------- CPU Timing ----------
start = time.time()

c_cpu = torch.matmul(a_cpu, b_cpu)

cpu_time = time.time() - start

print(f"\nCPU Time: {cpu_time:.4f} seconds")

# ---------- GPU Timing ----------
if torch.cuda.is_available():

    # Move matrices to GPU
    a_gpu = a_cpu.to(device)
    b_gpu = b_cpu.to(device)

    # Warm-up GPU (important)
    torch.matmul(a_gpu, b_gpu)

    torch.cuda.synchronize()

    start = time.time()

    c_gpu = torch.matmul(a_gpu, b_gpu)

    # Wait for GPU operations to finish
    torch.cuda.synchronize()

    gpu_time = time.time() - start

    print(f"GPU Time: {gpu_time:.4f} seconds")

    print(f"\nSpeedup: {cpu_time/gpu_time:.2f}x")

else:
    print("\nCUDA GPU not available")
```

Autograd

```
torch.tensor(2.0, requires_grad=True)
loss.backward()
optimizer.step()
optimizer.zero_grad() # x.grad.zero_()
```

- If $y = x^2$ ➔ $\frac{dy}{dx} = 2x$

```
def dy_dx(x):
    return 2 * x
```

- $y = x^2$
  $z = \sin(y)$

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$
$$= \cos(y) \,.\, 2x$$
$$= 2x\cos(x^2)$$

```
def dz_dx(x):
    return 2 * x * math.cos(x ** 2)
```

- $y = x^2$
  $z = \sin(y)$
  $u = e^z$

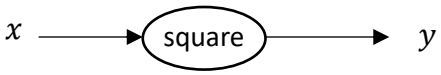
$$\frac{du}{dx} = \frac{du}{dz} \cdot \frac{dz}{dy} \cdot \frac{dy}{dx}$$
$$= e^z \cdot \cos(y) \cdot 2x$$
$$= 2x\, e^{\sin(x^2)} \cos(x^2)$$

Example1:

cgpa	placed
9.11	1
8.9	1
7	0
6.56	1
4.56	0


```
x = torch.tensor(3.0, requires_grad=True)
y = x**2
y # tensor(9, grad_fn=<pow3Backward>)

y.backward()
x.grad() # 6
```

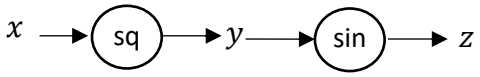


Example2:

$y = x^2$
$z = \sin(y)$

```
x = torch.tensor(3.0, requires_grad=True)
y = x**2
z = torch.sin(y)

z.backward()
x.grad() # -7
y.grad() ← Not a leaf node
```




```
x: tensor(3, requires_grad=True)
y: tensor(9, grad_fn = <PowBackward0>
z: tensor(0.4121, grad_fn = <SinBackward0>
```

PyTorch nn Module

Common Layers:
- nn.**Linear** (fully connected layer)
- nn.**Conv2d** (Convolutional layer)
- nn.**LSTM** (recurrent Layer) and many others

Loss Functions:
- nn.CrossEntropyLoss
- nn.MSELoss
- nn.NLLLoss

Embedding:
- nn.Embedding
- nn.MSELoss
- nn.NLLLoss

Normalization
- nn.BatchNorm2d
- nn.LayerNorm

Activation Functions:
- nn.ReLU
- nn.Sigmoid
- nn.Tanh

Container Modules:
- nn.ModuleList/Module
- nn.ParameterList/ParameterDict
- nn.Sequential: A sequential container to stack in order

Regularization and Dropout
- nn.Dropout
- nn.BatchNorm2d

Transformer
- nn.Transformer
- nn.TransformerEncoder/Decoder

Automate PyTorch Training Pipeline using nn Module

Create the NN Training Pipeline Manually:

Step1: Defining the Model

```
-----------------------------------------------------------------
class MySimpleNN():
  def __init__(self, inputs):
    self.weights = torch.rand(inputs.shape[1], 1,
requires_grad=True)
    self.bias = torch.zeros(1, requires_grad=True)

  def forward(self, inputs):
    z = torch.matmul(inputs, self.weights) + self.bias
    y_pred = torch.sigmoid(z)
    return y_pred

  def loss_function(self, y_pred, y):
    # Clamp predictions to avoid log(0)
    epsilon = 1e-7
    y_pred = torch.clamp(y_pred, epsilon, 1-epsilon)

    # Calculate loss
    loss = -(
          y * torch.log(y_pred) +
          (1 - y) * torch.log(1 - y_pred)
    )
    return loss.mean()
-----------------------------------------------------------------
Step3: Model Evaluation
-----------------------------------------------------------------
with torch.no_grad():
  y_pred = model.forward(x_test_tensor)
  y_pred = (y_pred > 0.5).float()

  accuracy = (y_pred == y_test_tensor).float().mean()
  print(f"Accuracy: {accuracy.item()}")
```

Step2: Training Pipeline
(forward --> loss --> backward --> parameterUpdate)

```
-----------------------------------------------------------------
model = MySimpleNN(x_train_tensor)

for epoch in range(epochs):
  # step1: forward pass
  y_pred = model.forward(x_train_tensor)

  # step2: loss calculation
  loss = model.loss_function(y_pred, y_train_tensor)

  # step3: backward pass
  loss.backward()

  # Step4: parameter update
  with torch.no_grad():
    model.weights -= learning_rate * model.weights.grad
    model.bias -= learning_rate * model.bias.grad

  #step5: zero gradients
  model.weights.grad.zero_()
  model.bias.grad.zero_()

  # print loss in each epoch
  print(f"Epoch: {epoch}, loss: {loss.item()}")
```

Automate using PyTorch nn Module

```
class MySimpleNN(nn.Module):
  def __init__(self, x_in, x_out, n_hidden):
    super().__init__()
    self.linear1 = nn.Linear(x_in, , n_hidden)
    self.relu = nn.ReLU()
    self.linear2 = nn.Linear(n_hidden, x_out)
    self.sigmoid = nn.Sigmoid()

  def forward(self, features):
    out = self.linear1(features)
    out = self.relu(out)
    out = self.linear2(out)
    out = self.sigmoid(out)
    return out
```

Automate all manual creation using Sequential

```
class MySimpleNNSeq(nn.Module):
  def __init__(self, in_features, out_features):
    super().__init__()
    self.network = nn.Sequential(
      nn.Linear(in_features, out_features),
      nn.ReLU(),
      nn.Linear(out_features, 1),
      nn.Sigmoid()
    )

  def forward(self, features):
    out = self.network(features)
    return out
```

```
model = MySimpleNNLinear(x_train_tensor.shape[1]).double()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

for epoch in range(epochs):
    y_pred = model(x_train_tensor)
    loss = nn.BCELoss()(y_pred,  y_train_tensor.view(-1, 1))

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    print(f"Epoch: {epoch}, loss: {loss.item()}")
```

Dataset & DataLoader

Why We need Dataset & DataLoader?

Lets take a scenario: **Train the model with Batch Gradient Descent**

```
# Create Model
model = MySimpleNN(x_train_tensor.shape[1]).double()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

for epoch in range(epochs):
  # Forward
  y_pred = model(x_train_tensor)
  loss = nn.BCELoss()(y_pred,  y_train_tensor)

  # Backward
  optimizer.zero_grad()
  loss.backward()
  optimizer.step()

  # print loss in each epoch
  print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}")
```

entire dataset in one go
→ that's batch gradient descent.

Issue with Batch Gradient Descent:

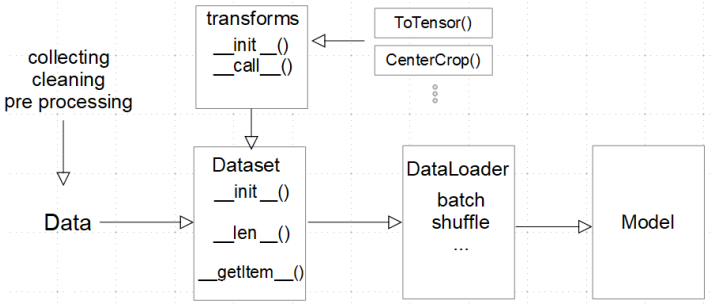
```
y_pred = model(x_train_tensor)
```

- **Memory inefficient:** Entire dataset must fit in memory
- **Slow updates:** Model updates only once per epoch

Optimization:
Mini-Batch Gradient Descent

DataLoader Control Flow

- At the start of each epoch, the DataLoader (if shuffle=True) shuffle indices (using sampler)
- It divides the indices into chunk, data samples are fetched from the Dataset object



- The samples are then collected and combined into a batch (using **collate_fn**)
- The batch is returned to the main training loop

```
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
  def __init__(self, features, labels):
    self.features = features
    self.labels = labels

  def __len__(self):
    return len(self.features)

  def __getitem__(self, idx):
    return self.features[idx], self.labels[idx]
```

```
# Create Custom Dataset
X = torch.arange(1,10)
y = 10 * X

# dataset = CustomDataset(x_train_tensor, y_train_tensor)
dataset = CustomDataset(X, y)
len(dataset)
dataset[:]
[
  [1,2,3,4,5,6,7,8,9],
  [10,20,30,40,50,60,70,80,90]
]
```

 # Initialize Dataloader

```
dataloader = DataLoader(dataset, batch_size=2, shuffle=False)
```

 # Use the Dataloader:

```
for batch_features, batch_labels in dataloader:
   print(batch_features, batch_labels)
```

```
tensor([1, 2]) tensor([10, 20])
tensor([3, 4]) tensor([30, 40])
tensor([5, 6]) tensor([50, 60])
tensor([7, 8]) tensor([70, 80])
tensor([9]) tensor([90])
```

Apply the DataLoader concept:

```
train_dataset = CustomDataset(x_train_tensor, y_train_tensor)
test_dataset = CustomDataset(x_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset,batch_size=32, shuffle=True)
```

```
# parameters
learning_rate = 0.1
epochs = 25

# Create Model
model = MySimpleNN(x_train_tensor.shape[1]).double()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

for epoch in range(epochs):
  for batch_features, batch_labels in train_loader: # (inputs, targets) or (X_batch, y_batch)
    # Forward
    y_pred = model(batch_features)
    loss = nn.BCELoss()(y_pred, batch_labels)
    # Backward
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

  # print loss in each epoch
  print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}")
```

Model evaluation using test_loader:

```
# Model evaluation using test_loader
model.eval() # set the model to evaluation mode
accuracy_list = []

with torch.no_grad():
  for inputs, targets in test_loader:
    # Forward
    y_pred = model(inputs)
    y_pred = (y_pred > 0.5).float()

    # calculate accuracy for current batch
    batch_accuracy = (y_pred == targets).float().mean()
    accuracy_list.append(batch_accuracy)

  # Calculate overall accuracy
  accuracy = torch.tensor(accuracy_list).mean()
  print(f"Test Accuracy: {accuracy * 100:.2f}%")
```